

Taller 01 — Foundation Models: Comparación, Decodificación y Razonamiento

Jaime Astudillo y Roberth Chachalo

MMIA 6013 — IA Generativa y Agentes, Universidad San Francisco de Quito

1 Tarea elegida

Elegimos diez problemas de aritmética con respuesta entera para verificar automáticamente los resultados. Agregamos tres casos contaminados y comparamos cómo cambian las respuestas al modificar el modelo, el prompt y el esfuerzo.

Los diez casos limpios y los tres contaminados, con sus respuestas esperadas, están en `datos/casos.json` del repositorio.

2 Parte 0 — El modelo base y la distribución

Fila `base_local`: `openai-community/gpt2`, 124M de parámetros, en CPU, sin API ni credenciales.

2.1 La distribución del siguiente token

En el dominio de números y cálculo usamos `As easy as one, two,` como frase hecha relativamente predecible y `To calculate the total, we need to` como contexto abierto. Los prefijos están en inglés, la lengua predominante del modelo base. La certeza se comprueba con la probabilidad máxima y la entropía; no se supone.

Se hace un forward por prefijo y se reutilizan sus logits para todas las temperaturas. La softmax, la entropía y el núcleo se calculan sobre **todo el vocabulario**, aunque la primera figura solo muestre los 15 primeros tokens.

Al bajar la temperatura, observamos más probabilidad concentrada en los primeros tokens. Al subirla, aumentaron la entropía y las alternativas. Con temperatura 1, el núcleo del 90 % incluyó cinco tokens en el prefijo seguro y 241 en el abierto.

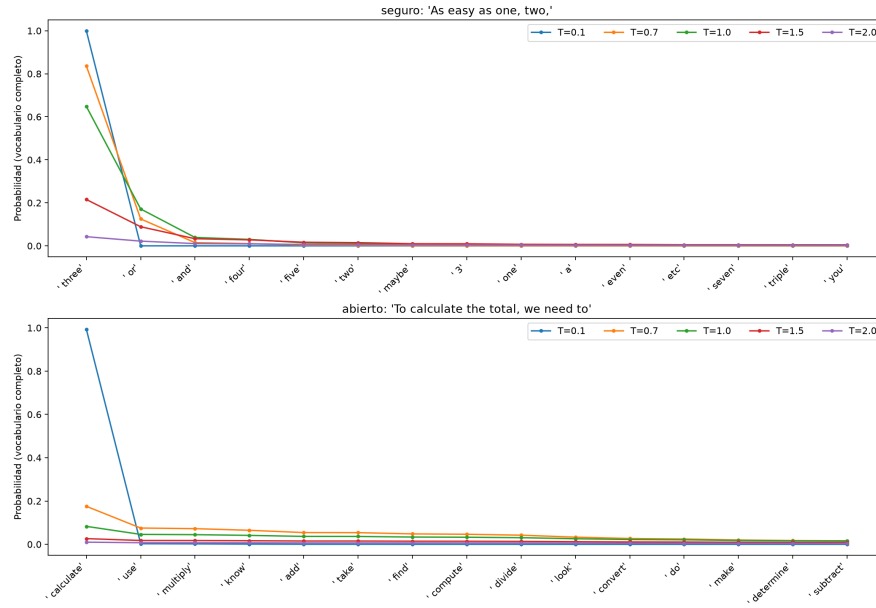


Figura 1. Distribución de los 15 candidatos más probables para $T \in \{0.1, 0.7, 1.0, 1.5, 2.0\}$ en los dos prefijos.

Tabla 1. Entropía y tamaño del núcleo del 90 % por prefijo y temperatura (GPT-2).

prefijo	T	entropía (bits)	núcleo 0.9	p máx
seguro	0.1	3.444e-05	1	1
seguro	0.7	0.8858	2	0.8365
seguro	1	2.323	5	0.6482
seguro	1.5	9.068	7659	0.2155
seguro	2	13.23	20702	0.04256
abierto	0.1	0.0684	1	0.9937
abierto	0.7	5.129	40	0.1755
abierto	1	7.1	241	0.08293
abierto	1.5	9.894	1735	0.02624
abierto	2	11.82	6464	0.009824

2.2 Las tres palancas sobre esa misma distribución

Usamos `model.generate(...)`, modo evaluación y semilla registrada por corrida. Para aislar cada palanca desactivamos los otros filtros: `top_k=0`, `top_p=1`, sin penalización de repetición y con un solo beam. El aviso de Transformers al enviar temperatura en modo greedy se conserva: ese parámetro se ignora. Véase la documentación de generación.

Cada generación se añade a `datos/resultados.csv`, con `parte=0` y costo cero. `datos/parte0_generaciones.jsonl` conserva además el prompt, los IDs, la semilla, la revisión del modelo, la fecha y los parámetros exactos. En esta parte `acierto` queda vacío porque no se evalúa exactitud.

Arriba se conservan las dos salidas del interruptor, las cinco corridas de `top_k=1` con sus IDs y soportes, la figura de cortes y la continuación de cien tokens sin editar. El conteo de cuatrigramas repetidos complementa la observación del texto; no evalúa su corrección.

Salida con `do_sample=False` (idéntica para $T = 0.2$ y $T = 1.5$):

```
calculate the total of all the items in the list.
```

```
$item = array( 'total' => $item, 'total' => $item, 'total'
=> $item, '
```

Con `do_sample=True` y `top_k=1`, las 25 corridas registradas en el CSV (5 ejecuciones del notebook con 5 semillas cada una) produjeron 1 salida distinta: idéntica a la de greedy (Tabla 2).

Tabla 2. Parte 0.b: cinco corridas con `do_sample=True` y `top_k=1`. IDs completos en `datos/parte0_generaciones.jsonl`; soporte/paso = tokens con probabilidad no nula tras el filtro.

corrida	semilla	primeros 10 IDs	tokens	soporte/paso	= greedy
0	123	15284 262 2472 286 477 262 3709 287 262 1351 ...	40	1	sí
1	124	15284 262 2472 286 477 262 3709 287 262 1351 ...	40	1	sí
2	125	15284 262 2472 286 477 262 3709 287 262 1351 ...	40	1	sí
3	126	15284 262 2472 286 477 262 3709 287 262 1351 ...	40	1	sí
4	127	15284 262 2472 286 477 262 3709 287 262 1351 ...	40	1	sí

Degeneración con 100 tokens en modo greedy (Holtzman et al. [1]), salida sin editar:

```
calculate the total of all the items in the list.
```

```
$item = array( 'total' => $item, 'total' => $item, 'total'
=> $item, 'total' => $item, 'total' => $item, 'total' =>
$item, 'total' => $item, 'total' => $item, 'total' =>
$item, 'total' => $item, 'total' => $item, 'total' => $
```

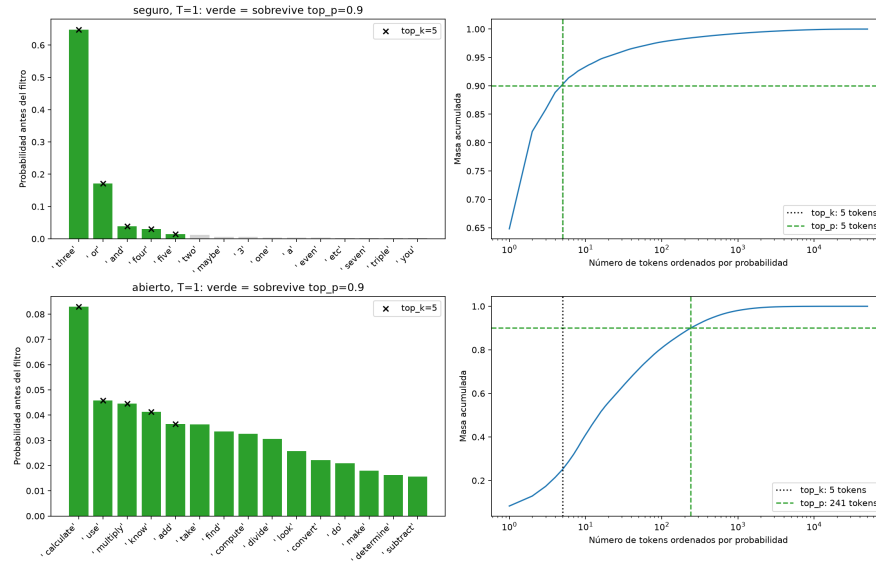


Figura 2. Dónde corta cada palanca: tokens que sobreviven a $\text{top_k}=5$ y a $\text{top_p}=0.9$ en cada prefijo. En el prefijo abierto los dos conjuntos no coinciden (5 frente a 241 tokens).

2.3 El límite del modelo base

Salida cruda de GPT-2 ante la instrucción exacta de las Partes 1 a 4:

Una cada una de 43 sillas. ¿Cuántos tornillos le sobran?

Una cada una de 43 sillas. ¿Cuántos tornillos le sobran?

Una cada una de 43 sillas. ¿Cuántos tornillos le sobran?

Una cada una de 43 sillas. ¿Cuántos

GPT-2 no entrega el entero pedido: copia un fragmento del enunciado («Una cada una de 43 sillas. ¿Cuántos tornillos le sobran?») y lo repite hasta agotar los 100 tokens, el mismo bucle de la degeneración de 0.b. No intenta calcular porque para él no existe la noción de pregunta: fue entrenado solo para predecir el siguiente token de un corpus, y lo que sigue estadísticamente a un problema escrito es más texto parecido. Se suma que el prompt está en español y su corpus es casi todo inglés. Lo que tienen los modelos de las Partes 1 a 4 y a este le falta es la etapa de alineación posterior al preentrenamiento, que enseña a tratar la entrada como una orden y a producir la forma de respuesta esperada. Esa etapa no garantiza que el número sea correcto; el verificador sigue siendo necesario.

3 Parte 1 — Comparación de modelos

Precios de la tabla semestral, cada uno con la fecha de verificación de su propia fila (Tabla 3).

Tabla 3. Parte 1: tres modelos sobre los 10 casos. Latencia = tiempo de pared de la petición.

modelo	exactitud	lat. (s)	tok. in	tok. out	costo USD	verificado
gpt-4o-mini	0	1.9063	793	12	0.00013	2026-08-26
gpt-5.6-luna	1	1.13	783	405	0.00064	2026-09-18
qwen3:1.7b	0.9	10.677	1011	7424	0	local

Elegiríamos gpt-5.6-luna: acertó los diez casos, fue el más rápido (1,13 s frente a 1,91 s de gpt-4o-mini y 10,7 s de qwen3 en local) y costó 0,00064 USD por los diez. Qwen3 acertó nueve a costo cero, pero requiere recursos locales y tarda porque razona antes de responder; lo consideraríamos para trabajo local o con datos que no puedan salir de la máquina. Observaciones cualitativas: gpt-4o-mini respetó el formato (un solo número, sin unidades) en los diez casos, no fue verboso (1,2 tokens de salida por llamada) y falló los diez: dio cifras equivocadas con seguridad, sin señal de duda. gpt-5.6-luna respetó el formato y no mostró errores ni respuestas vacías. qwen3 respetó el formato en la respuesta visible, pero generó 7 424 tokens de salida en diez casos, casi todos de razonamiento; su único fallo fue de cálculo, no de formato. Ninguno devolvió errores de API. La latencia es tiempo de pared de la petición completa, red incluida. Ampliaríamos el conjunto antes de decidir en producción: diez casos permiten comparar, no generalizar.

Confirmamos que los precios se leyeron de la tabla semestral del enunciado, con la fecha de cada fila, y no de memoria.

4 Parte 2 — Decodificación

4.1 Matriz de exposición

Cada parámetro se mandó con dos valores extremos, 3 casos $\times$ 2 corridas. El estado se decide por la dispersión entre corridas de la tupla (salida, tokens de salida), no por la respuesta final (Tabla 4); los mensajes de error se transcriben completos a continuación de la tabla.

Tabla 4. Parte 2.a: matriz de exposición, declarado frente a observado (2026-09-20). Evidencia: porcentaje de casos cuyas corridas difieren con cada valor; o número de llamadas con error.

modelo	parámetro	declarado (tabla)	observado	evidencia
gpt-4o-mini	temperature	si	aceptado y actual	temperature=0: 0% / temperature=1.5: 100%
gpt-4o-mini	top_p	si	aceptado y actual	top_p=0.1: 33% / top_p=1: 67%
gpt-4o-mini	top_k	no	rechazado	12/12 llamadas con error
gpt-5.6-luna	temperature	no_en_esta_fila	rechazado	12/12 llamadas con error
gpt-5.6-luna	top_p	no_en_esta_fila	rechazado	6/12 llamadas con error
gpt-5.6-luna	top_k	no	rechazado	12/12 llamadas con error
qwen3:1.7b	temperature	si	aceptado y actual	temperature=0: 33% / temperature=1.5: 100%
qwen3:1.7b	top_p	si	aceptado y actual	top_p=0.1: 33% / top_p=1: 100%
qwen3:1.7b	top_k	si	aceptado y actual	top_k=1: 33% / top_k=40: 100%

Mensajes literales completos de las celdas rechazadas: gpt-4o-mini / top_k:

```
TypeError: Completions.create() got an unexpected keyword argument 'top_k'
```

gpt-5.6-luna / temperature:

```
400: Error code: 400 - {'error': {'message': "Unsupported value: 'temperature' does not support 0.0 with this model. Only the default (1) value is supported.", 'type': 'invalid_request_error', 'param': 'temperature', 'code': 'unsupported_value'}}
```

gpt-5.6-luna / top_p:

```
400: Error code: 400 - {'error': {'message': "Unsupported parameter: 'top_p' is not supported with this model.", 'type': 'invalid_request_error', 'param': 'top_p', 'code': 'unsupported_parameter'}}
```

gpt-5.6-luna / top_k:

```
TypeError: Completions.create() got an unexpected keyword argument 'top_k'
```

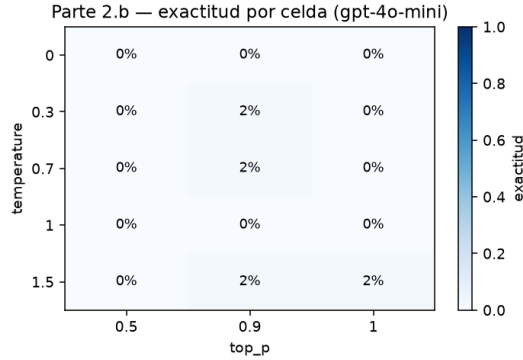
En gpt-4o-mini, temperatura y top-p mostraron cambios entre corridas. Top-k falló en el SDK, antes de llegar al servidor. Luna rechazó las temperaturas probadas y aceptó top_p=1, pero rechazó 0.1. En Ollama observamos contenido en thinking sin pedirlo, contrario al valor predeterminado indicado en la tabla.

4.2 Barrido de temperature y top-p

Estimación previa con el supuesto del enunciado (1000 tokens de entrada y 600 de salida por llamada) al precio de gpt-4o-mini verificado el 2026-08-26: 750 llamadas $\approx$ 0.38 USD. Se corrieron las 750 (15 celdas $\times$ 5 corridas $\times$ 10 casos); costo real 0.033 USD. Catorce configuraciones promediaron entre 1,18 y 1,36 tokens de salida; la restante (T=1,5, top_p=1) promedió 771,14. La media global fue de 52,6 tokens por llamada, frente a los 600 del supuesto.

Tabla 5. Parte 2.b: rejilla temperature $\times$ top_p sobre gpt-4o-mini, 5 corridas $\times$ 10 casos por celda (750 llamadas).

T	top_p	exactitud	tokens out (media)	estabilidad
0.000	0.500	0.000	1.200	0.960
0.000	0.900	0.000	1.200	0.960
0.000	1.000	0.000	1.240	0.980
0.300	0.500	0.000	1.340	0.740
0.300	0.900	0.020	1.240	0.820
0.300	1.000	0.000	1.240	0.740
0.700	0.500	0.000	1.360	0.640
0.700	0.900	0.020	1.180	0.560
0.700	1.000	0.000	1.240	0.560
1.000	0.500	0.000	1.280	0.640
1.000	0.900	0.000	1.220	0.420
1.000	1.000	0.000	1.340	0.480
1.500	0.500	0.000	1.260	0.580
1.500	0.900	0.020	1.340	0.440
1.500	1.000	0.020	771.140	0.280

**Figura 3.** Exactitud por celda de la rejilla (gpt-4o-mini).**Tabla 6.** Parte 2.b: barrido de top_k en local (qwen3:1.7b, T=1, 5 corridas secuenciales $\times 10$ casos).

top_k	exactitud	tokens out (media)	estabilidad	casos idénticos x5
1.000	0.920	1080.320	0.960	8
5.000	0.960	947.780	0.880	7
40.000	0.980	931.120	0.860	7

La temperatura divide los logits antes del softmax: con $T < 1$ amplía la distancia entre el token más probable y el resto y concentra la masa; con $T > 1$ la reduce y reparte masa hacia la cola. top_p corta esa cola por masa acumulada. Sobre gpt-4o-mini (750 llamadas), la exactitud fue 0 % en once celdas y 2 % (una de cincuenta) en las otras cuatro: el muestreo no cambia lo que el modelo sabe hacer, solo cuánto varía su salida. Lo que sí se mueve es la estabilidad: entre 0,96 y 0,98 con $T=0$, alrededor de 0,75 con $T=0,3$ y 0,28 en $T=1,5$ con top_p=1. Esa celda además degenera: 771 tokens de salida en promedio frente a 1,2-1,4 en el resto, sin corte de núcleo la cola aplanada no termina; con top_p=0,5 la misma $T=1,5$ vuelve a 1,26 tokens. Que $T=0$ no dé 1,0 es un hallazgo: la rama argmax no es una promesa de reproducibilidad en una API. Para esta tarea elegiríamos $T=0$ y top_p=1, por estabilidad, ya que ninguna celda mejora la exactitud: la única palanca que la mueve es el prompt o el modelo. Para tres titulares alternativos elegiríamos $T=1$ con top_p=0,9: hay dispersión y el núcleo descarta tokens de menor probabilidad. En local, top_k=1 dio la misma salida en ocho de diez casos en cinco corridas secuenciales, no en los diez, y la dispersión creció con k (estabilidad 0,96 $\rightarrow$ 0,86 de top_k=1 a 40) y la exactitud observada pasó de 92 % a 98 %.

5 Parte 3 — Prompting estructurado

Las cuatro plantillas están en `src/prompts.py`. Los ejemplos del few-shot son problemas inventados, no los diez casos. La variante estructurada usa el modo JSON del proveedor (`response_format` con tipo `json_object`), sin esquema estricto.

Tabla 7. Parte 3: cuatro variantes de prompting sobre gpt-4o-mini, 10 casos.

variante	exactitud	tokens in	tokens out	costo USD
zero_shot	0.1	793	11	0.00013
few_shot	0	1842	11	0.00028
cot	1	933	3160	0.00204
structured	0	883	61	0.00017

Respuestas completas de cada variante sobre el mismo caso (las diez por variante están en el CSV, columna `salida`):

zero_shot (caso c01, esperado 311, extraído 245.0, acierto no):

245

few_shot (caso c01, esperado 311, extraído 81.0, acierto no):

81

cot (caso c01, esperado 311, extraído 311.0, acierto sí):

Para resolver este problema, primero vamos a determinar cuántos tornillos hay en las cajas y luego cuántos tornillos se usan para fabricar las sillas.

1. ****Calcular el total de tornillos comprados:****
 - El taller compra 17 cajas.
 - Cada caja contiene 36 tornillos.
 - Por lo tanto, el total de tornillos comprados es:


```

          \[
          17 \text{ cajas} \times 36 \text{ tornillos/caja} =
          612 \text{ tornillos}
          \]
```
2. ****Calcular cuántos tornillos se usan para las sillas:****
 - Se usan 7 tornillos para cada silla.
 - El taller necesita hacer 43 sillas.
 - Por lo tanto, el total de tornillos usados es:


```

          \[
          7 \text{ tornillos/silla} \times 43 \text{ sillas} =
```

```

301 \text{ tornillos}
    \]

3. **Calcular cuántos tornillos sobran:**
    - Para encontrar cuántos tornillos sobran, restamos los
    tornillos usados del total de tornillos comprados:
    \[
    612 \text{ \text{ tornillos} } - 301 \text{ \text{ tornillos} } = 311
    \text{ \text{ tornillos} }
    \]

```

Finalmente, el número de tornillos que le sobran al taller es:

Respuesta: 311

structured (caso c01, esperado 311, extraído 71.0, acierto no):

```
{"respuesta": 71}
```

Zero-shot acertó un caso y few-shot ninguno. En estos casos, pedir directamente un número produjo menor exactitud que solicitar pasos intermedios; los ejemplos duplican los tokens de entrada (1 842 frente a 793 en los diez casos) sin ganar nada, porque enseñan el formato, no el procedimiento. Chain-of-Thought pasó a diez de diez a cambio de 3 160 tokens de salida, casi 300 veces los 11 de zero-shot, y un costo 16 veces mayor (0,00204 frente a 0,00013 USD): compra cómputo intermedio, que es lo que esta tarea necesita. La variante estructurada usó el modo JSON del proveedor (response_format con json_object), sin esquema estricto: las diez salidas parsearon como JSON y ninguna fue correcta. Distinguimos lo que garantiza el proveedor de lo que verificamos nosotros: el modo JSON garantiza la forma; el valor lo comprobó nuestro verificador. Usaríamos CoT cuando la tarea tenga pasos dependientes y la exactitud pese más que el costo y la latencia; zero-shot cuando la respuesta sea de recuperación directa o una etiqueta, donde razonar no añade información; few-shot cuando lo difícil sea el formato o el criterio de clasificación, no el cálculo; y salida estructurada siempre que otro programa consuma la respuesta, combinada con CoT si el contenido lo exige, porque por sí sola solo garantiza que parsea. No asumiríamos que combinar técnicas mejora el resultado sin medir esa combinación con los mismos diez casos.

6 Parte 4 — Modelos de razonamiento y niveles de esfuerzo

6.1 Barrido de esfuerzo

Modelo gpt-5.6-luna. El dial es `reasoning_effort`. El contador de tokens de razonamiento es el campo `reasoning_tokens` que devuelve la API. Se corrió

low completo antes de los otros dos: 388 tokens de salida totales en 10 casos (38,8 por llamada, razonamiento incluido) frente a los 1 500 por llamada que supone el presupuesto del enunciado para el nivel bajo; la cuenta se rehizo con esa cifra antes de seguir.

Tabla 8. Parte 4.a: barrido de esfuerzo en gpt-5.6-luna, 10 casos por nivel.

esfuerzo	exactitud	tok. razonamiento	tok. visibles	lat. (s)	costo USD
low	1	287	101	1.2317	0.00062
medium	1	288	101	0.95785	0.00062
high	1	375	101	1.0125	0.00073

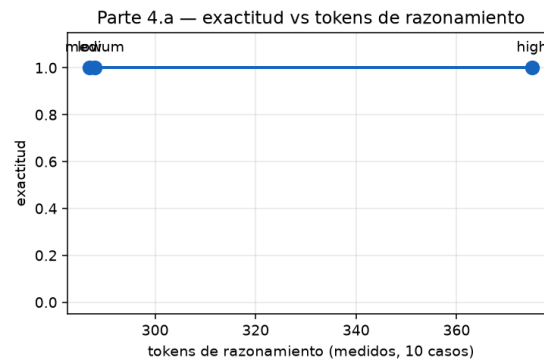
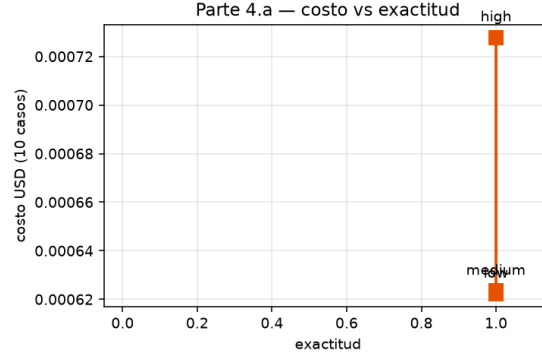


Figura 4. Exactitud frente a tokens de razonamiento medidos.

**Figura 5.** Costo en USD frente a exactitud.

La curva es plana: diez de diez en low, medium y high. Los tokens de razonamiento medidos fueron 287, 288 y 375 en los diez casos. Low y medium tuvieron consumos casi iguales en esta ejecución. No encontramos una mejora de exactitud al aumentar el esfuerzo, pero necesitaríamos más repeticiones para evaluar su efecto. High sí mueve el contador (+30 %) y el costo, de 0,00062 a 0,00073 USD, sin ganar un punto de exactitud: pagamos 0,0001 USD adicionales por cero puntos, así que no podemos calcular un costo por punto. Low fue suficiente entre los niveles probados; no evaluamos niveles inferiores. Las cifras también corrigen el supuesto del presupuesto: 38,8 tokens de salida totales por llamada en low frente a los 1 500 que supone el enunciado, 38,7 veces menos; el costo estimado con ese supuesto era 0,020 USD y el real fue 0,00062 USD, 32,1 veces menos. La lección no es que el dial no sirva, sino que el nivel se elige por tarea y se mide: estos diez casos están por debajo del umbral de dificultad donde el esfuerzo empieza a pagar. Con casos más difíciles y varias corridas por nivel esperaríamos ver la curva.

6.2 El caso donde pensar más hace daño

Casos c11 (distracción por lo irrelevante), c12 (sobreajuste al marco) y c13 (correlación espuria), según los modos de fallo de Gema et al. [2].

Tabla 9. Parte 4.b: los tres casos contaminados en los niveles extremos.

esfuerzo	caso	acierto	tokens razonamiento
high	c11	1	0
high	c12	1	24
high	c13	1	26
low	c11	1	0
low	c12	1	0
low	c13	1	21

No observamos empeoramiento: ambos niveles acertaron los tres casos. No reproducimos los fallos descritos por Gema et al., pero eso no contradice su estudio. Tres casos no permiten establecer una regla general.

7 Parte 5 — Reflexión teórica

7.1 Explica intuitivamente qué hace el mecanismo de atención en un transformer. ¿Por qué es importante el concepto de query, key, value?

Entendemos la atención como una forma de decidir qué partes del contexto aportan información para representar cada token. Cada posición produce tres vectores por proyección lineal: la query expresa lo que esa posición busca, la key permite que las demás posiciones se comparen con esa búsqueda y el value contiene la información que se mezcla. El modelo calcula puntuaciones entre la query y todas las keys, las convierte en pesos con softmax y obtiene una combinación ponderada de los values. Así relaciona palabras cercanas o distantes según el contexto, y todas las posiciones se calculan en paralelo, sin el cuello de botella secuencial de una red recurrente. Separar los tres roles permite aprender por separado qué comparar y qué contenido recuperar. En generación causal, una máscara impide atender a tokens futuros.

7.2 ¿Cuál es la diferencia entre un modelo base y un modelo alineado? ¿Por qué un modelo base no es directamente útil para las aplicaciones de este curso?

Salida completa de 0.c ante la instrucción de la tarea, sin editar:

Una cada una de 43 sillan. ¿Cuántos tornillos le sobran?

Una cada una de 43 sillan. ¿Cuántos tornillos le sobran?

Una cada una de 43 sillan. ¿Cuántos tornillos le sobran?

Una cada una de 43 sillan. ¿Cuántos

En nuestra prueba, GPT-2 continuó el enunciado sin resolverlo. Lo relacionamos con la diferencia entre continuar texto y estar ajustado para seguir instrucciones. Un modelo base aprende patrones del corpus durante el preentrenamiento y solo predice el siguiente token. Un modelo alineado recibe después ejemplos de peticiones y respuestas, y a menudo entrenamiento con preferencias humanas; eso le enseña a tratar la entrada como una orden y a responder en la forma esperada. Para las aplicaciones del curso necesitamos que la salida responda a una tarea y respete un formato, algo que no podemos dar por hecho con el modelo base. Aun así, el modelo base nos resultó útil para observar directamente los logits y el muestreo. El ajuste mejora el comportamiento, pero no elimina la necesidad de verificar las respuestas.

7.3 En la Parte 3, la variante con Chain-of-Thought probablemente usó más tokens que la zero-shot. ¿Cuándo vale la pena pagar ese costo y cuándo no?

En nuestros resultados, CoT pasó de un acierto a diez frente a zero-shot, con los tokens de salida de 11 a 3 160 y el costo de 0,00013 a 0,00204 USD por los diez casos. Pagaríamos esa diferencia cuando la respuesta dependa de pasos intermedios que una instrucción directa no resolvió, cuando el costo de un error supere con mucho el de los tokens y cuando la latencia adicional sea aceptable. No la pagaríamos para extraer un dato explícito o devolver una etiqueta, donde primero probaríamos una instrucción corta; ni en aplicaciones de alto volumen con margen pequeño por llamada. También compararíamos con un modelo de razonamiento: en la Parte 4, gpt-5.6-luna resolvió los mismos casos con 38,8 tokens de salida por llamada en low, frente a 316 de CoT en gpt-4o-mini. Gastar más tokens solo tiene sentido si la mejora medida lo justifica.

7.4 En tu matriz de la Parte 2.a hay al menos una celda «rechazado». ¿Qué gana un proveedor al retirar un parámetro de muestreo de su API, y qué pierde quien lo usaba?

En nuestras pruebas, gpt-5.6-luna rechazó con 400 los valores de temperature y top_p distintos del predeterminado. Consideramos que el proveedor gana control sobre la distribución de salida de un modelo cuyo comportamiento depende de una fase de razonamiento: con muestreo libre el usuario podría degradar esa fase y el proveedor tendría que dar soporte a resultados que no puede reproducir. Es una explicación plausible, no una intención que podamos demostrar con el CSV. Quien usaba la palanca pierde la capacidad de intercambiar exactitud por diversidad en la misma llamada y la reproducibilidad aproximada de $T=0$. Lo que la sustituye es el nivel de esfuerzo: en vez de controlar cómo se muestrea, se controla cuánto se piensa. La diversidad, cuando hace falta, se obtiene pidiendo varias alternativas en el prompt o con varias llamadas, y no equivale a la temperatura.

7.5 Los tokens de razonamiento de la Parte 4 se facturan y casi nunca se devuelven en crudo. Si tu modelo te mostrara la traza completa, ¿sería eso una explicación de cómo llegó a la respuesta?

No tomaríamos una traza completa como explicación suficiente de cómo se obtuvo la respuesta. Lanham et al. (2023) miden si la respuesta cambia cuando se altera la cadena de razonamiento y encuentran que en muchos casos el modelo llega al mismo resultado con una traza truncada o con errores insertados, y que esa fidelidad varía con el tamaño del modelo y con la tarea. Una explicación puede ser coherente sin reflejar el proceso que produjo la respuesta. Para auditar un sistema en producción, esto significa que la traza es evidencia de proceso, no

prueba de causa: sirve para detectar errores visibles y para medir cuánto cómputo se gastó, como el contador de la Parte 4, pero no basta para certificar por qué el modelo respondió lo que respondió. La auditoría debe apoyarse en pruebas de comportamiento sobre casos verificables y registro de versiones y condiciones de ejecución.

8 Reproducibilidad y desviaciones declaradas

- Código, casos, `requirements.txt` y `datos/resultados.csv` (1159 filas, una por llamada, las locales con costo cero) en <https://github.com/jastk45/Maestria->.
Carpeta: `IA-Generativa-y-Agentes/semana1/taller01_foundation_models`
- Las claves se leen de un `.env` excluido por `.gitignore`; ninguna aparece en el informe, el notebook ni el repositorio.
- Las llamadas a OpenAI (Partes 1, 2.a, 2.b, 3 y 4) se mandaron con 12 peticiones en paralelo (`scripts/ejecutar_partes.py`); las de Ollama en la Parte 1 y las sondas 2.a, con 4 (`scripts/ejecutar_local.py`); el barrido de `top_k` en local, en secuencia. Que $T = 0$ no dé estabilidad 1,0 podría deberse a la agrupación de peticiones en el servidor; no lo comprobamos.
- Los tokens de razonamiento de qwen3 se miden en tokens: `eval_count` de Ollama (todo lo generado) menos los tokens de la respuesta visible contados con el tokenizador de Qwen3. Los de gpt-5.6-luna salen del contador `reasoning_tokens` de la API.

References

1. Holtzman, A., Buys, J., Du, L., Forbes, M., Choi, Y.: The Curious Case of Neural Text Degeneration. ICLR (2020). <https://arxiv.org/abs/1904.09751>
2. Gema, A.P., et al.: Inverse Scaling in Test-Time Compute. TMLR (2025)
3. Lanham, T., et al.: Measuring Faithfulness in Chain-of-Thought Reasoning. arXiv:2307.13702 (2023)
4. Vaswani, A., et al.: Attention Is All You Need. NeurIPS (2017)
5. Radford, A., et al.: Language Models are Unsupervised Multitask Learners. OpenAI (2019). Model card: <https://huggingface.co/openai-community/gpt2>